PRACTICAL - 6

Implementation of Matrix Chain Multiplication using Dynamic Programming

Aim of the Practical:

To find the parenthesization with minimum scalar multiplications using dynamic programming and derive its complexity.

Recurrence:

$$m[i,j] = 0, \text{ if } i = j$$

$$m[i,j] = \min\{i \le k < j\} [m[i,k] + m[k+1,j] + p[i-1]p[k]p[j]]$$

Program:

```
#include <iostream>
#include <vector>
#include <climits>
using namespace std;
long long matrixChain(const vector<int>& p) {
    int n = p.size() - 1;
    vector<vector<long long>> dp(n + 1, vector<long long>(n + 1, 0));
    for (int len = 2; len <= n; len++) {
        for (int i = 1; i <= n - len + 1; i++) {
            int j = i + len - 1;
            dp[i][j] = LLONG_MAX;
            for (int k = i; k < j; k++) {
                long long cost = dp[i][k] + dp[k + 1][j]
                               + 1LL * p[i - 1] * p[k] * p[j];
                if (cost < dp[i][j]) dp[i][j] = cost;
            }
        }
    }
    return dp[1][n];
}
int main() {
    vector<int> p = {4, 10, 3, 8};
    cout << matrixChain(p);
}
```

Output:

```
216
```

Step-by-Step Cost Calculation for the Sample:

- A1 = 4×10, A2 = 10×3, A3 = 3×8.
- (A1A2)A3: cost = (4×10×3) + (4×3×8) = 120 + 96 = 216.
- A1(A2A3): cost = (10×3×8) + (4×10×8) = 240 + 320 = 560.
- Minimum = 216 scalar multiplications.

Time Analysis:

Derivation of Time Complexity:

Let n be the number of matrices.

The outer loop chooses chain length len from 2 to n: O(n) possibilities.

For a fixed len, the starting index i can take at most O(n) values.

For every pair (i,j), the split point k runs from i to j-1: at most O(n) choices.

$$T(n) = \Sigma len\ \Sigma i\ \Sigma k\ O(1)$$

$$T(n) = O(n \cdot n \cdot n) = O(n^3)$$

More precisely, the number of evaluated split points is:

$$\Sigma(len = 2 \text{ to } n)\ (n - len + 1)(len - 1)$$

Name: SHAIK ASHWAK BASHA Enrollment No: 92460118522

which grows on the order of n^3.

The DP table has $(n + 1)^2$ entries.

$$S(n) = \Theta(n^2)$$

Final Time Analysis:

- Time Complexity: $\Theta(n^3)$.
- Space Complexity: $\Theta(n^2)$.

Result:

Matrix Chain Multiplication was solved successfully. The sample minimum cost is 216 and the DP complexity is cubic in the number of matrices.